

Scalable RevOps Architecture: HubSpot to Google Sheets Data Pipeline

Role: Junior CRM & Business Process Automation Engineer

Tech Stack: Make.com, HubSpot CRM API, Google Apps Script (JavaScript), Google Sheets (SQL-style QUERY)

1. Executive Summary & Objective

To eliminate manual data entry and ensure real-time sales visibility, I engineered an end-to-end automation that pulls deal data from HubSpot CRM, strictly from the Sales Pipeline, and filters for Closed Won deals only. The system dynamically enriches each record with associated contact and company data, processes the payloads through a JavaScript normalization layer, and powers a fully automated monthly performance dashboard.

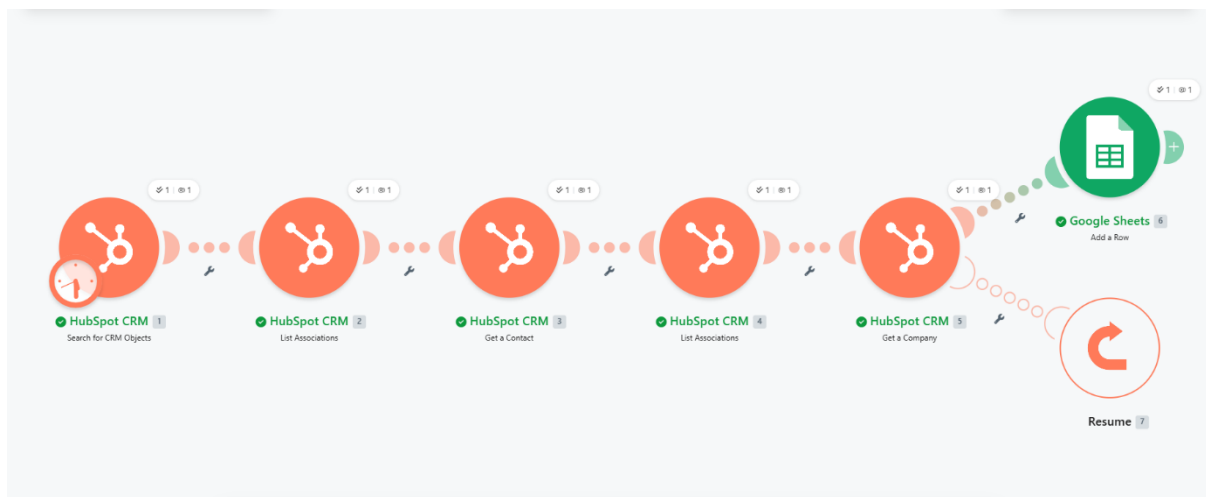


Figure 1: Macro Architecture: The end-to-end Make.com scenario routing HubSpot data to Google Sheets.

2. Phase 1: API Routing & Association Extraction

The initial challenge was extracting nested relational data from HubSpot's backend. In HubSpot, deals, contacts, and companies are separate objects, requiring dedicated API calls beyond basic field mapping.

The Technical Hurdle:

When requesting associated contacts and companies, the HubSpot API returns arrays, not pure strings. Passing a raw array into a "Get CRM Record" module causes a 404 Object Not Found error because the API strictly requires a numeric ID.

The Engineered Solution: Instead of relying on Make.com's default formula wrapper (which often passes data as literal text strings), I utilized native array indexing. By mapping `2. Associations.Contacts[1]: Associated Object ID`, I forced the system to bypass the array

container and extract the pure numeric ID of the primary contact, ensuring zero API crashes even when multiple contacts are attached to a deal.

HubSpot CRM

> **Connection ***

My HubSpot CRM connect... Add

For more information on how to create a connection to HubSpot CRM, see the [online Help](#).

> **Contact ID *** ☒ Map

2. Associations.Contacts[]: Associated Object ID

> **Output Properties** ☐ Map

phone

Contact properties to retrieve. Large requests can result in a **414 Request-URI Too Large** error. If you receive this error, select fewer properties.

> **Parse Custom Fields** ☐ Map

☒ Yes ☐ No

Enables the automatic parsing of custom fields. If some of the fields are parsed incorrectly (i.e. numbers parsed as dates), you can deactivate the automatic parsing.

☒ Advanced settings Cancel Save

Figure 2: *Bypassing formula UI for pure native array indexing to extract raw numeric IDs.*

3. Phase 2: Data Normalization (Google Apps Script)

Webhook payloads deliver raw, machine-readable data (e.g., timestamps like 2026-06-30T18... and raw numeric IDs for Deal Stages). To make this data usable, I deployed a Google Apps Script layer to handle data formatting and date processing.

The Script's Core Functions:

- **Data Type Enforcement:** Webhooks occasionally pass numeric values (like deal amounts) as text strings, which breaks dashboard aggregation. The script strictly parses these payloads into integers.
- **Dynamic Formatting:** Automatically extracts the Month and Year from the raw ISO timestamp for accurate dashboard grouping.
- **Redundancy Protection:** Implemented an array-mapping duplicate checker that scans existing Deal Names before appending new rows, ensuring data integrity if a webhook misfires or loops.
- **Normalization:** Normalized deal owner names to prevent mismatches in aggregations.

```

function processMakeData() {
  const ss = SpreadsheetApp.getActiveSpreadsheet();
  const rawSheet = ss.getSheetByName("Raw_Data");
  const cleanSheet = ss.getSheetByName("Clean_Data");

  // Fetch all data
  const rawData = rawSheet.getDataRange().getValues();
  const cleanData = cleanSheet.getDataRange().getValues();

  // Create an array of existing Deal Names to prevent duplicates
  const existingDeals = cleanData.map(row => row[0]);

  // Dictionaries for Normalization
  const ownerMap = {
    "91639591": "Manan Asim" // Add other owner IDs here if needed
  };

  const stageMap = {
    "987832630": "Closed Won"
  };

  let newRowsToAppend = [];

  // Loop through raw data (skip header row)
  for (let i = 1; i < rawData.length; i++) {
    let row = rawData[i];
    let dealName = row[0];

    // Protect against duplicate entries
    if (dealName === "" || existingDeals.includes(dealName)) {
      continue;
    }

    // Convert close date into a properly formatted date value
    let rawDate = new Date(row[2]);
    let formattedDate = Utilities.formatDate(rawDate, "GMT", "yyyy-MM-dd");

    // Extract Month and Year
    let month = Utilities.formatDate(rawDate, "GMT", "MMMM");
    let year = Utilities.formatDate(rawDate, "GMT", "yyyy");

    // Normalise deal owner names and deal stages
    let rawStage = String(row[3]);
    let cleanStage = stageMap[rawStage] || rawStage;

    let rawOwner = String(row[4]);
    let cleanOwner = ownerMap[rawOwner] || rawOwner;

    // Build the cleaned row
    let cleanRow = [
      dealName,           // A: Deal Name
      Number(row[1]),     // B: Amount
      formattedDate,      // C: Close Date
      cleanStage,         // D: Stage
      cleanOwner,         // E: Owner
      row[5],            // F: Pipeline
      row[6],            // G: Phone
      row[7],            // H: Company
      month,             // I: Month
      year               // J: Year
    ];

    newRowsToAppend.push(cleanRow);
    existingDeals.push(dealName); // Add to check-list to prevent duplicates in same run
  }

  // Batch append new rows for maximum efficiency
  if (newRowsToAppend.length > 0) {
    cleanSheet.getRange(cleanSheet.getLastRow() + 1, 1, newRowsToAppend.length,
      newRowsToAppend[0].length).setValues(newRowsToAppend);
  }
}

```

Figure 3: JavaScript normalization layer processing raw webhook payloads and enforcing data types.

4. Phase 3: The Automated Reporting Dashboard

To fulfill the business requirement of a clear monthly view of Closed Won performance, I architected a multi-tab database structure separating Raw_Data from the executive Dashboard.

Using a Google Sheets QUERY function (=QUERY(Clean_Data!A:J, "SELECT J, I, E, COUNT(A), SUM(B)...")), the dashboard automatically groups the normalized data by Year and Month, counts the total deals, and calculates total revenue by deal owner.

A1	=QUERY(Clean_Data!A:J, "SELECT J, I, E, COUNT(A), SUM(B) WHERE A IS NOT NULL GROUP BY J, I, E ORDER BY J DESC, I DESC LABEL J 'Year', I 'Month', E 'Deal Owner', COUNT(A) 'Total Won Deals', SUM(B) 'Total Revenue'")														
	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	Year	Month	Deal Owner	Total Won Deals	Total Revenue										
2	2026	June	Manan Asim	1	5000										
3															
4															
5															
6															
7															
8															
9															
10															
11															
12															
13															
14															
15															

Figure 4: Real-time performance dashboard powered by automated QUERY aggregations and conditional color scaling.

5. Business Impact & ROI

- **Zero Manual Data Entry:** Replaced hours of manual spreadsheet updates with a real-time, event-driven pipeline.
- **Bulletproof Data Integrity:** By combining Make.com's targeted extraction with Apps Script's strict type-casting, the system is immune to webhook payload shifts and user input errors.
- **Scalable Operations:** The architecture is designed for growth. As the sales team expands, the dynamic ID mapping and QUERY logic will seamlessly aggregate new reps without requiring codebase updates.